

Nouveau processus de publication des documents de travail

OFCE

Note à l'équipe — août 2026

Pourquoi ce changement ?

L'ancien système permettait à n'importe quel dépôt GitHub de revendiquer n'importe quel numéro de WP lors de son premier déploiement, sans autorisation préalable. Il était également possible de contourner la protection existante en choisissant un numéro de version personnalisé. Ces deux failles pouvaient conduire à l'écrasement accidentel (ou malveillant) d'un document de travail déjà publié sur le site de l'OFCE.

La solution retenue est un **registre central** (ofceweb/wp-registry) dans lequel un administrateur valide explicitement l'association entre un numéro de WP et le dépôt GitHub qui a le droit de le publier. Aucun dépôt ne peut publier un WP numéroté sans figurer dans ce registre.

Les trois états d'un document de travail

Un WP passe désormais par trois états successifs.

1. Brouillon initial

Le dépôt est créé et configuré avec `setup_wp()`, mais aucune demande de numéro n'a encore été soumise. Le rendu produit une prévisualisation sur **GitHub Pages** (<https://{org}.github.io/{nom-du-depot}/>). C'est un lien temporaire, à usage interne, qui s'écroule à chaque nouveau rendu.

2. Staging (en attente de validation)

L'auteur·e a transféré la propriété de son wp à l'organisation OFCE et se prépare à demander un numéro via `wp_registry_request()`. Une pull request est ouverte dans le registre central, en attente d'approbation par un administrateur web. En attendant, chaque rendu dépose le document sur le **serveur FTP de l'OFCE dans un espace de staging** (si `stage-target: ftp`, voir plus bas) à l'adresse :

```
https://www.ofce.fr/staging/{nom-du-depot}/v1/
```

Contrairement à GitHub Pages, cet espace est **versionné** : chaque version envoyée à des relecteurs (v1, v2, v3...) reste accessible à son URL propre. Les relecteurs peuvent donc conserver leurs liens même si une nouvelle version est déposée. Les versions de staging sont conservées indéfiniment jusqu'à décision explicite de l'administrateur.

Des commentaires sont faits, jusqu'à la validation éditoriale et technique. La requête de numéro peut être validée lorsque le wp est validé.

3. Publié

La pull request dans le registre a été fusionnée par un administrateur. Lors du rendu suivant, le document est automatiquement déployé sur le chemin numéroté définitif :

```
https://www.ofce.fr/wp/{annee}/{numero}/{version}/
```

Le numéro et l'année sont désormais **attribués par le registre**, non par l'auteur-e. La redirection vers l'URL stable (sans segment de version) est gérée automatiquement.

Ce que fait l'auteur-e, étape par étape

1. <code>setup_wp()</code>	Initialise le dépôt (une fois)
2. [rédaction du WP]	
3. <code>render_wp()</code>	Rendu + prévisualisation locale
4. <code>deploy_wp()</code>	→ cible lue depuis <code>stage-target</code> (gh-pages par défaut)
– quand le WP est prêt pour la numérotation –	
5. <code>wp_registry_request()</code>	Demande un numéro (ouvre une PR dans le registre)
6. <code>render_wp()</code>	Rendu
7. <code>deploy_wp()</code>	→ cible toujours lue depuis <code>stage-target</code>
– après approbation admin et fusion de la PR –	
8. <code>render_wp()</code>	Détecte automatiquement l'enregistrement
9. <code>deploy_wp()</code>	→ FTP production (url numérotée définitive)

Les étapes 6–7 peuvent être répétées autant de fois que nécessaire pendant la relecture (chaque `wp_version_up()` incrémente la version et crée une nouvelle URL de staging).

Important (changement depuis la v0.9.4) : `deploy_wp()` n'a plus de paramètre `target`. La destination (avant publication) est désormais lue directement dans `_quarto.yml`, clé `stage-target`: `gh-pages` ou `stage-target: ftp`. Pour changer de cible, relancer :

```
setup_wp(stage_target = "ftp")      # ou "gh-pages"
```

`setup_wp()` écrit systématiquement cette clé (valeur par défaut `gh-pages` pour un nouveau WP) et recalcule `website.site-url` en conséquence. Une fois le WP confirmé dans le registre (`stage = FALSE`), `stage-target` est ignoré et le déploiement va toujours vers FTP production.

Décisions clés et leurs raisons

Le registre est dans `ofceweb/wp-registry`, pas dans `ofce`

Le registre est un dépôt **public**, ce qui permet aux workflows GitHub Actions de le lire sans avoir besoin d'un secret ou d'un token. Il est hébergé dans l'organisation `ofceweb` (distincte

de `ofce`) pour séparer les responsabilités : `ofce` héberge les dépôts des WP, `ofceweb` héberge l'infrastructure web.

L'appartenance à l'organisation `ofce` est obligatoire pour publier

Les secrets FTP de production (`WP_USER`, `WP_PASSWORD`) sont stockés au niveau de l'organisation `ofce`. Un dépôt hors de cette organisation n'y a simplement pas accès — la protection est donc structurelle, pas seulement procédurale. La demande de numéro (`wp_registry_request()`) est également bloquée si le dépôt n'est pas dans `ofce`.

Le **rendu local** fonctionne sans restriction d'organisation : un auteur peut rédiger et prévisualiser son WP depuis n'importe quel dépôt, puis transférer la propriété vers `ofce` avant de demander un numéro.

Le staging utilise des credentials FTP distincts avec `chroot`

Les secrets de staging (`STAGING_USERNAME`, `STAGING_PASSWORD`) sont différents des secrets de production. Le compte FTP de staging est configuré côté serveur avec un **`chroot` sur le dossier `stage/`** : même si un dépôt mal configuré tentait d'écrire hors de son espace de staging, le serveur l'en empêcherait. La sécurité est garantie côté serveur, pas seulement par les workflows.

La vérification du registre a lieu au moment du rendu, pas du déploiement

`render_wp()` consulte le registre **avant** de lancer Quarto. Cela permet : - d'injecter un bandeau « Version provisoire — non publiée » dans le HTML, le PDF et le Typst produits (géré par les extensions `ofce-quarto-extensions`) ; - d'écrire le champ `stage` dans `manifest.json`, que `deploy_wp()` lit ensuite pour choisir la bonne destination (staging ou production).

Le workflow `ftp_deploy.yml` effectue sa propre vérification indépendante au moment du déploiement, comme deuxième ligne de défense.

Le routage avant publication vient du YAML, pas d'un paramètre de fonction

Avant la v0.9.4, `deploy_wp(target = ...)` choisissait la destination à l'appel. Ce paramètre a été supprimé : la destination est maintenant une propriété persistante du dépôt (`stage-target` dans `_quarto.yml`), au même titre que `annee` ou `wp`. Avantage : un `render_wp()` + `deploy_wp()` lancé en CI ou par une autre personne va toujours au même endroit, sans dépendre d'un argument oublié.

`check_wp()` vérifie la connexion GitHub

Un nouveau diagnostic (`gh:login`) confirme que `gh` est authentifié avant de tenter une opération de staging ou de registre — ces deux opérations échouent silencieusement sans authentification GitHub valide.

La citation n'est pas requise pour le rendu

Le champ `citation` dans `_quarto.yml` est calculé automatiquement par `setup_wp()` à partir de `annee` et `wp`. Il n'est pas pertinent en mode staging (pas encore de numéro) et n'est jamais un bloquant pour le rendu : son absence génère un avertissement invitant à relancer `setup_wp()`.

Sécurité : le détail des protections

Le schéma repose sur plusieurs couches indépendantes, pour qu’une faille dans l’une n’entraîne pas la publication non autorisée d’un WP. Certaines sont implémentées dans ce package (`ofceweb`, vérifiables dans le code), d’autres sont des réglages GitHub côté dépôt `ofceweb/wp-registry` (gouvernance — décrits ici tels que décidés, mais à confirmer périodiquement puisqu’ils ne sont pas visibles depuis ce dépôt).

Couche 1 — Les secrets FTP de production ne sortent pas de l’organisation `ofce`

`WP_USER`/`WP_PASSWORD` (production) sont des secrets **au niveau de l’organisation `ofce`**. Un dépôt hébergé ailleurs (compte personnel, autre organisation) n’y a structurellement pas accès : même s’il exécutait `ftp_deploy.yml`, l’étape FTP échouerait faute de secrets. Cette protection ne dépend d’aucune ligne de code — elle vient de la configuration GitHub des secrets eux-mêmes.

`wp_registry_request()` ajoute un contrôle **côté client**, avant même d’ouvrir une pull request : le remote `origin` du dépôt est résolu et la fonction s’arrête si le propriétaire n’est pas `ofce`. Cela évite d’ouvrir une PR qu’un-e admin devrait de toute façon rejeter, mais ce n’est qu’un confort — la vraie garantie reste l’absence de secrets hors `ofce`.

Couche 2 — Vérification anti-collision au moment du déploiement

`ftp_deploy.yml` contient une étape dédiée (« Vérification anti-collision ») qui s’exécute **avant** l’envoi FTP : elle télécharge le `manifest.json` déjà présent à l’URL cible et compare son champ `source-repo` à `github.repository` (le dépôt qui exécute le workflow). Si les deux diffèrent, le déploiement est annulé avec `::error::` plutôt que d’écraser le WP existant. Un premier déploiement (pas encore de `manifest.json` distant) ou un manifeste antérieur à ce contrôle (pas de champ `source-repo`) laissent passer, sans bloquer les cas légitimes.

Cette vérification compare l’état **actuellement publié à ce chemin précis** — elle ne protège donc pas, à elle seule, contre un dépôt qui choisirait un chemin jamais encore utilisé (par exemple un numéro de WP jamais publié). Ce deuxième trou est ce que ferme la couche 3.

Couche 3 — Le registre central comme autorisation préalable

Le registre (`registry.json` dans `ofceweb/wp-registry`) ferme la faille laissée ouverte par la couche 2 : il ne suffit plus de choisir un chemin inoccupé, il faut une **entrée validée par un-e admin pour le couple {année, wp}** avant de pouvoir publier dessus.

- `render_wp()` interroge le registre (lecture publique, non authentifiée, via `raw.githubusercontent.com`) à chaque rendu et détermine `stage` (publié si le `source-repo` du dépôt local a une entrée `type: "repo"` correspondante, staging sinon). Si le registre est inaccessible (réseau, panne), le résultat par défaut est `stage = TRUE` — repli **sûr** vers le staging plutôt qu’une publication non vérifiée.
- Seule une **pull request fusionnée** dans `wp-registry` crée une entrée valide. `wp_registry_request()` ouvre la PR automatiquement mais n’attend pas sa fusion (fire-and-forget) : la décision de fusionner reste un acte humain et délibéré.
- Le registre étant public, sa lecture ne nécessite aucun token — seule son écriture (fusion de PR) est protégée.

État du registre : à jour pour 2025 et 2026

Le registre a été complété le 27/08/2026 (PR #6) avec les WP historiques `pdf-only` de 2025 et 2026, à partir de `ofcweb/webhome/publications/working_papers.yml` : 35 entrées ajoutées, en plus des 4 entrées `repo` déjà présentes (2025/23, 2026/6, 2026/9, 2026/10). **Le registre est désormais la source de vérité pour la numérotation des WP 2025 et 2026** — toute vérification d'un numéro (déjà publié, disponible, ou son type `repo/pdf-only`) doit se faire dans `registry.json`, plus dans `working_papers.yml` qui n'est qu'un affichage dérivé.

Chaque entrée `pdf-only` porte désormais un champ `pdf-path` (chemin relatif à `webhome`, ex. `pdf/dtravail/OFCEWP2025-01.pdf`), vérifié en HTTP avant ajout. Une seule exception reste en dehors du registre : **WP 2025/24** (« La reprise d'emploi est-elle toujours rémunératrice ? », Allègre & Pucci), actuellement publié hors `ofce` (GitHub Pages personnel) sans PDF associé — en attente du transfert annoncé du dépôt vers l'organisation `ofce`, après quoi il pourra être enregistré comme `type: "repo"`.

Couche 4 — Gouvernance du registre lui-même

Ce qui empêche une PR malveillante ou erronée d'être fusionnée dans `wp-registry`. **Vérifié directement sur le dépôt le 27/08/2026** (API GitHub, branche `main`) :

- **CODEOWNERS confirmé** (`.github/CODEOWNERS`) :

```
*           @xtimbeau
/registry.json @xtimbeau
```

`xtimbeau` est bien désigné relecteur requis, y compris spécifiquement pour `registry.json`. Aucune équipe `wp-admins` n'existe encore — conforme au statut documenté (« admin actuel : `xtimbeau` seul »).

- **Protection de branche confirmée** sur `main` : revue de pull request obligatoire (`required_approving_review_count: 1`) avec `require_code_owner_reviews: true` — une PR ne peut être fusionnée sans l'approbation d'un CODEOWNERS. `allow_deletions` est désactivé.
- **Un écart persistant** : `enforce_admins: false` — l'administrateur (compte avec droits admin sur le dépôt) peut techniquement contourner la revue obligatoire et pousser directement sur `main`. Sans conséquence pratique tant que `xtimbeau` est seul admin *et* seul relecteur, mais à revisiter si le rôle s'élargit.
- **CI de validation ajoutée le 27/08/2026** (PR #3, fusionnée) : `main` contient désormais `registry.schema.json` (schéma JSON draft-07) et `.github/workflows/validate-registry.yml`. Ce workflow, déclenché sur chaque pull request touchant `registry.json/registry.schema.json`, effectue trois contrôles : JSON bien formé (`jq empty`), conformité au schéma (`ajv-cli`, qui impose entre autres le pattern `^ofce/.+` sur `source-repo` pour les entrées `type: "repo"`), et une vérification `jq` explicite redondante pour un message d'erreur lisible. Testé localement avant fusion : accepte les 4 entrées existantes, rejette une entrée fictive `eviluser/hijack`. **Déclaré *required status check* le 27/08/2026** (`required_status_checks.strict: true`, check `Validate registry.json`, épinglé à l'app

GitHub Actions) : une PR touchant `registry.json` ne peut désormais plus être fusionnée si ce contrôle échoue — y compris pour un-e admin, `enforce_admins: false` ne couvrant que le nombre de revues requises, pas les status checks. La couche 4 est donc désormais complète.

Ces réglages vivent dans la configuration GitHub du dépôt `wp-registry`, pas dans le code du package `ofceweb` — à revérifier périodiquement si la gouvernance évolue (nouvelle équipe d’admins, ajout d’une CI, etc.).

Couche 5 — Isolation du staging FTP par chroot serveur

Le staging (avant enregistrement) utilise des identifiants FTP **distincts** (`STAGING_USER/STAGING_PASSWORD`) des identifiants de production. Le compte FTP de staging est configuré côté serveur OFCE avec un **chroot** sur `www/staging/` : même en cas de dépôt mal configuré ou de valeur incorrecte dans `FTP_STAGING_DIR`, le serveur physiquement ne permet pas d’écrire en dehors de cet espace — il ne peut donc pas y avoir d’écrasement accidentel d’un WP publié via le chemin de staging. Cette garantie est indépendante du code des workflows : elle tient à la configuration du serveur FTP lui-même.

Chiffrement optionnel (staticrypt)

Indépendamment de ce qui précède, `STATICRYPT_PASSWORD` (si défini comme secret) déclenche un chiffrement du site statique avant l’envoi FTP, pour les deux workflows (`ftp_deploy.yml` et `ftp_stage.yml`). C’est un contrôle d’accès en lecture pour des relecteurs externes, orthogonal aux protections contre l’écrasement/l’usurpation décrites ci-dessus.

Résumé

Couche	Protège contre	Où c’est implémenté
Secrets scopés à <code>ofce</code>	Publication depuis un dépôt hors organisation	Configuration GitHub (secrets d’organisation)
Anti-collision (manifest)	Écrasement d’un WP déjà publié au même chemin	<code>ftp_deploy.yml</code> (étape dédiée)
Registre central	Revendication d’un numéro jamais encore publié, sans autorisation	<code>render_wp()</code> (lecture) + <code>wp-registry</code> (écriture)
Gouvernance du registre	Fusion d’une entrée invalide ou malveillante	Branch protection + <code>CODEOWNERS</code> dans <code>wp-registry</code>
Chroot FTP staging	Écriture hors de l’espace de staging	Configuration du serveur FTP OFCE
Chiffrement staticrypt	Accès en lecture par des tiers non autorisés	<code>ftp_deploy.yml</code> / <code>ftp_stage.yml</code> (si secret défini)

Pour les administrateurs web

La seule action requise de la part de l'administration est de **fusionner (ou rejeter) les pull requests** ouvertes dans `ofcweb/wp-registry`. Chaque PR contient le nom du dépôt demandeur, l'année, le numéro proposé et l'adresse de contact de l'auteur-e. Un CI vérifie que le numéro demandé n'est pas déjà pris avant d'autoriser la fusion.

L'administrateur actuel du registre est **xtimbeau**. Une équipe `wp-admins` dans l'organisation `ofcweb` est prévue pour élargir ce rôle si nécessaire.

Évolutions récentes (v0.10.1 - août 2026)

Alignement du tooling WP et prévisions

Les fonctions de rendu et déploiement des **prévisions** (`render_prev()`, `stage_prev()`, `publish_prev()`) ont été alignées sur le pattern WP pour une cohérence maximale :

- **Suppression du paramètre `check_repo`** de `render_prev()` et des orchestrateurs (`stage_prev()`, `publish_prev()`, `render_prev_publish()`) : le rendu Quarto ne dépend pas de l'état du dépôt git. La vérification git-status existait avant mais entraînait en conflit avec les workflows CI (qui clonent le dépôt mais ne le configurent pas comme dépôt git "actif"). Supprimée pour cohérence avec `render_wp()`.
- **Vérification GitHub unifiée** : un nouveau helper `check_gh_login()` (partagé entre WP et prévisions) appelle `gh::gh("GET /user")` pour vérifier la connexion GitHub. Intégré dans :
 - `setup_prev()` au démarrage (alerte précoce si authentification manquante)
 - `stage_prev()` et `publish_prev()` (empêche les déploiements staging/registry muets en cas d'absence de token)
 - `check_prev()` (diagnostic `gh:login`, identique à `check_wp()`)
 - `check_wp()` (refactorisé pour utiliser le helper partagé)
- **Workflows toujours force-remplacés** : la politique a changé pour `setup_wp()` (était : créer seulement si absent) → toujours force-remplacer depuis le package template. `setup_prev()` était déjà ainsi. Raison : les workflows ne doivent **jamais** être édités manuellement — ce sont des templates de référence du package. Un force-replace garantit que chaque exécution de `setup_wp()` / `setup_prev()` propage les corrections critiques (comme les secrets mal nommés, les chemins `.staticcrypt` mal ciblés) automatiquement au prochain `setup_*` exécuté.

Correctifs dans les templates

- **`ftp_deploy_staging.yml`** (prévisions) : secret GitHub Actions mal nommé (`STAGING_USERNAME` → `STAGING_USER`). Décorrélation FTP échouait en CI faute du secret correct. Corrigé.
- **`ftp_deploy_staging.yml` et `ftp_stage.yml`** (tous deux) : la commande `staticcrypt` visait `./` (le répertoire courant) au lieu de `./*` (son contenu). Résultat : le répertoire lui-même était chiffré plutôt que ses fichiers. Corrigé.

Ces corrections sont désormais dans les templates du package. Au prochain `setup_wp()` / `setup_prev()`, tous les dépôts recevront automatiquement les workflows corrigés.

Ce processus est implémenté dans le package R `ofcweb`, version v0.10.1 (mise à jour v0.9.4 → v0.10.1, août 2026). Voir [NEWS.md](#) pour le détail de chaque version.